Probabilistic Machine Learning

Lecture 06: Evaluation and Empirical Risk Minimisation

Prof. Dr. Michael Granitzer

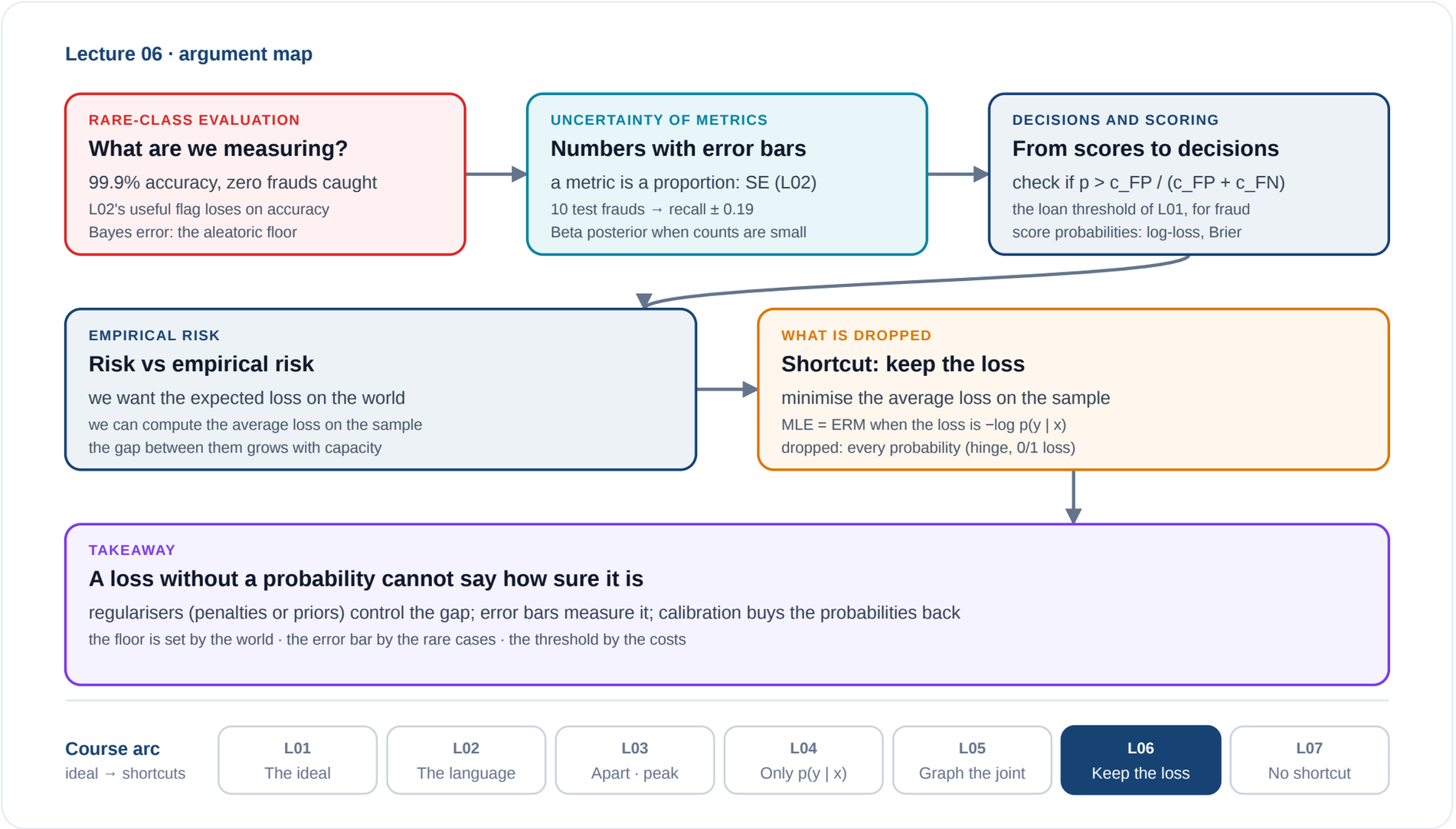


Speaker notes

The lecture uses the fraud model of Lecture 02 throughout (prevalence 1/1,000, recall 99%, false-positive rate 1%). Evaluating Rare-Class Classifiers defines the true error and the Bayes error, the aleatoric floor of Lecture 01, and shows that on a rare class accuracy favours the model that catches nothing. Uncertainty of Metrics treats every metric as an estimate, with a standard error or a Beta posterior (Lecture 02). Decisions and Scoring Rules derives the decision threshold $c_{FP}/(c_{FP} + c_{FN})$ from the costs, as in Lecture 01, and scores the probabilities with log-loss and the Brier score. Empirical Risk Minimisation defines risk and empirical risk: training by ERM keeps a loss and returns probabilities only when that loss is a negative log-probability; a regulariser corresponds to a prior (MAP), and calibration on held-out data recovers probabilities from scores.

Cross-validation schemes and precision/recall are assumed from earlier machine learning courses and are recapped in one slide each.

Argument Map



Speaker notes

Evaluating Rare-Class Classifiers: on a rare class accuracy rewards the model that catches nothing, and every classifier has a floor, the Bayes error. Uncertainty of Metrics: every metric is an estimate; for a rare class its error bar is set by the number of rare cases. Decisions and Scoring Rules: a calibrated probability plus the costs determines the decision, and the probabilities themselves can be scored. Empirical Risk Minimisation: training minimised an average loss on the sample (the empirical risk); unless that loss was a negative log-probability, the output carries no probability, and calibration on held-out data is needed to recover one — the reason the decision rule and the scoring rules of the decisions module require calibrated probabilities.

Evaluating Rare-Class Classifiers

Guiding Question: A fraud model reports 99.9% accuracy and has never caught a single fraud. What should we have measured instead — and how low could the error of the best model be?

What We Want to Know

The **true error** of a classifier h is the probability that it errs on a fresh example drawn from the world:

$$\mathrm{err}^*(h) = P\big(h(X) \neq Y\big), \qquad (X, Y) \sim p$$

The **Bayes error** is the smallest true error any classifier can reach:

$$\min_h \mathrm{err}^*(h) = \mathbb{E}_X\Big[\min\big(p(y{=}1 \mid X),\ 1 - p(y{=}1 \mid X)\big)\Big]$$

A transaction type is fraudulent 55% of the time. Could a better model push its error below 45%?

Speaker notes

The Bayes-optimal rule predicts, for every x, the label with the larger conditional probability. On that x it errs exactly with the probability of the other label, $\min\big(p(y{=}1 \mid x), 1 - p(y{=}1 \mid x)\big)$; averaging over $p(x)$ gives the formula. The error is not zero because Y given $X = x$ is still random: identical attribute vectors carry different labels.

For the transaction type that is fraudulent 55% of the time (the gift-card score of Lecture 01), the best rule predicts "fraud" and is wrong 45% of the time; no model using the same attributes does better. The floor is relative to the attributes: a new attribute (device, location) that separates frauds from legitimate cases of that type can lower it, more rows with the same attributes cannot. More data only moves a learned classifier's error towards the floor.

The true error is an expectation under the unknown p and cannot be computed. What can be computed is the error on a finite sample, an estimate whose precision is the subject of Uncertainty of Metrics.

What We Want to Know

The **true error** of a classifier h is the probability that it errs on a fresh example drawn from the world:

$$\mathrm{err}^*(h) = P\big(h(X) \neq Y\big), \qquad (X, Y) \sim p$$

The **Bayes error** is the smallest true error any classifier can reach:

$$\min_h \mathrm{err}^*(h) = \mathbb{E}_X\Big[\min\big(p(y{=}1 \mid X),\ 1 - p(y{=}1 \mid X)\big)\Big]$$

A transaction type is fraudulent 55% of the time. Could a better model push its error below 45%?

No — only **new attributes** can. The Bayes error is aleatoric uncertainty (Lecture 01's identical patients) written as an error rate: the floor that no amount of data removes.

Lecture 02's Flag, Evaluated

per 100,000 transactions, 100 frauds	always "legitimate"	Lecture 02's flag
frauds caught · honest customers flagged	0 · 0	99 · 999
accuracy	**99.9%**	**99.0%**
recall $P(\text{flag} \mid \text{fraud})$	0%	99%
precision $P(\text{fraud} \mid \text{flag})$	—	9%

Which model would you deploy — and which one does accuracy recommend?

Speaker notes

The table applies the flag of Lecture 02 to 100,000 transactions with prevalence 1/1,000: recall 99% and a false-positive rate of 1% (999 of 99,900 honest customers). The flag decides 99 + 98,901 = 99,000 cases correctly (99.0%); "always legitimate" decides the 99,900 honest cases correctly (99.9%).

Recall and precision are the two conditionals of Bayes' theorem: $P(\text{flag} \mid \text{fraud}) = 99/100 = 0.99$ and $P(\text{fraud} \mid \text{flag}) = 99/1{,}098 \approx 0.09$. Precision is undefined for the always-legitimate model, which flags nothing (0/0).

A constant majority predictor has accuracy $1 - \pi$ for minority prevalence π. Another classifier beats it on accuracy only if it makes fewer than πn errors in total — fewer than 100 here. The flag makes 1,000 errors (1 missed fraud, 999 false alarms) and loses on accuracy although it catches 99 of 100 frauds. Accuracy weights both kinds of error equally, which is appropriate only if both cost the same.

Lecture 02's Flag, Evaluated

per 100,000 transactions, 100 frauds	always "legitimate"	Lecture 02's flag
frauds caught · honest customers flagged	0 · 0	99 · 999
accuracy	**99.9%**	**99.0%**
recall $P(\text{flag} \mid \text{fraud})$	0%	99%
precision $P(\text{fraud} \mid \text{flag})$	—	9%

Which model would you deploy — and which one does accuracy recommend?

> By accuracy, the model that catches **nothing** wins. On a rare class, accuracy counts the majority and hides that a model is useless.

What to Report Instead

- → **A baseline** next to every number: always-majority for classification, the mean for regression. "99.9%" is not a result until you know that doing nothing scores 99.9%.
- → **Measures that condition on the rare class**: recall conditions on the truth, precision on the decision — the two directions of Bayes' theorem that Lecture 02 warned not to swap.
- → **The costs**: which of the two mistakes the application pays for. The decisions module turns them into a threshold.

> Precision depends on the base rate, recall does not: the same flag has precision 9% at 1 in 1,000 and about 50% at 1 in 100.

Speaker notes

A baseline makes a number interpretable: the always-majority classifier has accuracy $1 - \pi$; for regression, predicting the training mean gives a mean squared error equal to the variance of y (the reference point of R^2).

Recall $P(\text{flag} \mid \text{fraud})$ and the false-positive rate $P(\text{flag} \mid \text{legitimate})$ condition on the truth and are properties of the classifier alone. Precision conditions on the decision and, by Bayes' theorem, mixes them with the base rate π:

$$\text{precision} = \frac{\text{recall} \cdot \pi}{\text{recall} \cdot \pi + \text{FPR} \cdot (1 - \pi)}$$

At $\pi = 1/100$ on 100,000 transactions: 1,000 frauds, 990 caught; 99,000 honest customers, 1% of them (990) flagged; precision $990/1{,}980 = 0.5$, recall still 0.99. At $\pi = 1/1{,}000$ the same formula gives $0.00099/0.01098 \approx 0.09$. Precision therefore changes when an unchanged model is moved to a market with a different fraud rate.

Costs make the two kinds of error comparable; Decisions and Scoring Rules derives the threshold $c_{FP}/(c_{FP} + c_{FN})$ from them.

Mini-Exercise

A model flags **every** transaction as fraud (one in 1,000 is fraudulent).

What are its recall and its precision?



Speaker notes

Flagging everything gives TP = all frauds and FN = 0, so recall is 1. Every transaction is flagged, so precision is the share of frauds among all transactions — the base rate, 0.1%. Accuracy equals the base rate as well, because only the frauds are classified correctly.

The always-flag and always-legitimate models are the two ends of a threshold sweep (threshold 0 and threshold 1). On a rare class one maximises recall, the other accuracy, and neither is useful. Each single measure can be maximised by a trivial model; recall is therefore reported together with precision or the false-positive rate, and the operating point is chosen from the costs. The F_1 score, the harmonic mean of precision and recall, is about 0.002 for the always-flag model; it combines the two by a fixed convention rather than by cost.

Mini-Exercise

A model flags **every** transaction as fraud (one in 1,000 is fraudulent).

What are its recall and its precision?

Recall **100%** — it catches every fraud. Precision **0.1%** — the base rate. (Accuracy: 0.1%.)
Recall alone can be bought by flagging everything; that is why it is always reported together with precision, or with the costs.



Takeaway

We want the **true error**, whose floor — the **Bayes error** — is aleatoric uncertainty that no data removes. On a rare class, accuracy measures the majority: report a **baseline** and measures that condition on the rare class.

Uncertainty of Metrics

Guiding Question: The test set says recall 0.90. Would you promise the bank that the model catches 90% of next month's frauds?

Where an Error Number Comes From

estimate	how	what it is
training error	evaluate on the data the model was fitted on	optimistically biased — on average below the true error of the fitted model, often far below
hold-out	split once; fit on one part, evaluate on the other	an unbiased estimate of err^* — for **one** split, with a sampling error
k-fold cross-validation	each of k folds serves once as the test set; average	uses every example for testing once; still an estimate from a sample

> A test set that is consulted for choices — which model, which threshold — becomes training data. Keep one set that is looked at once.

Speaker notes

Training error is optimistically biased because the model was chosen to fit those data: its expected value is below the expected true error of the fitted model. It is not a lower bound for a single sample — on a given split it can exceed the true error. For a model fixed without looking at the test data, the test-set error is an unbiased estimate of its true error.

Hold-out is one split; it is not 2-fold cross-validation, in which each half serves once as the test set. k-fold cross-validation estimates the performance of the *training procedure* on training sets of size $(k-1)n/k$, not of one particular fitted model. Its fold estimates are correlated because the training sets overlap, so the naive standard error of the fold average is too small.

The warning describes adaptive overfitting: every choice made by looking at test results (model, hyperparameters, threshold) fits that choice to the noise of the test set, as on public leaderboards. A validation set (or nested cross-validation) for the choices and a test set evaluated once keep the final estimate unbiased.

A Metric Is a Proportion

Accuracy on n test cases is the mean of n yes/no indicators — a Bernoulli sample (Lecture 02). Its standard error is $\sigma/\sqrt{n}$ with $\sigma^2 = p(1-p)$:

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

A test set of **10,000** transactions, one in 1,000 fraudulent:

0.990 ± 0.002

accuracy: $n = 10{,}000$, $\mathrm{SE} \approx 0.001$

0.90 ± 0.19

recall, 9 of 10 frauds caught: $n = 10$, $\mathrm{SE} \approx 0.095$

Same test set, same model — why is one error bar a hundred times wider?

Speaker notes

Accuracy is $\hat{p} = \frac{1}{n}\sum_i 1[h(\mathbf{x}_i) = y_i]$, the mean of n Bernoulli(p) indicators, so $\mathrm{Var}(\hat{p}) = p(1-p)/n$; the standard error plugs in $\hat{p}$. SE(accuracy) $= \sqrt{0.99 \cdot 0.01/10{,}000} \approx 0.001$; SE(recall) $= \sqrt{0.9 \cdot 0.1/10} \approx 0.095$. The cards show ±1.96 SE, rounded: ±0.002 and ±0.19, a ratio of about 95.

Recall is a proportion over the frauds only, so its n is the number of test frauds (10), not the 10,000 transactions; precision likewise uses the number of flagged cases. The size of the test set fixes only the error bar of accuracy.

The normal interval for the recall, $[0.71, 1.09]$, extends beyond 1: with $n = 10$ and $\hat{p}$ near the boundary, the normal approximation to the binomial fails. Intervals that stay inside $[0, 1]$ are the Wilson interval ($[0.60, 0.98]$ here), the exact Clopper–Pearson interval ($[0.55, 1.00]$) and the Beta posterior of the next slide.

A Metric Is a Proportion

Accuracy on n test cases is the mean of n yes/no indicators — a Bernoulli sample (Lecture 02). Its standard error is $\sigma/\sqrt{n}$ with $\sigma^2 = p(1-p)$:

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

A test set of **10,000** transactions, one in 1,000 fraudulent:

0.990 ± 0.002

accuracy: $n = 10{,}000$, $\mathrm{SE} \approx 0.001$

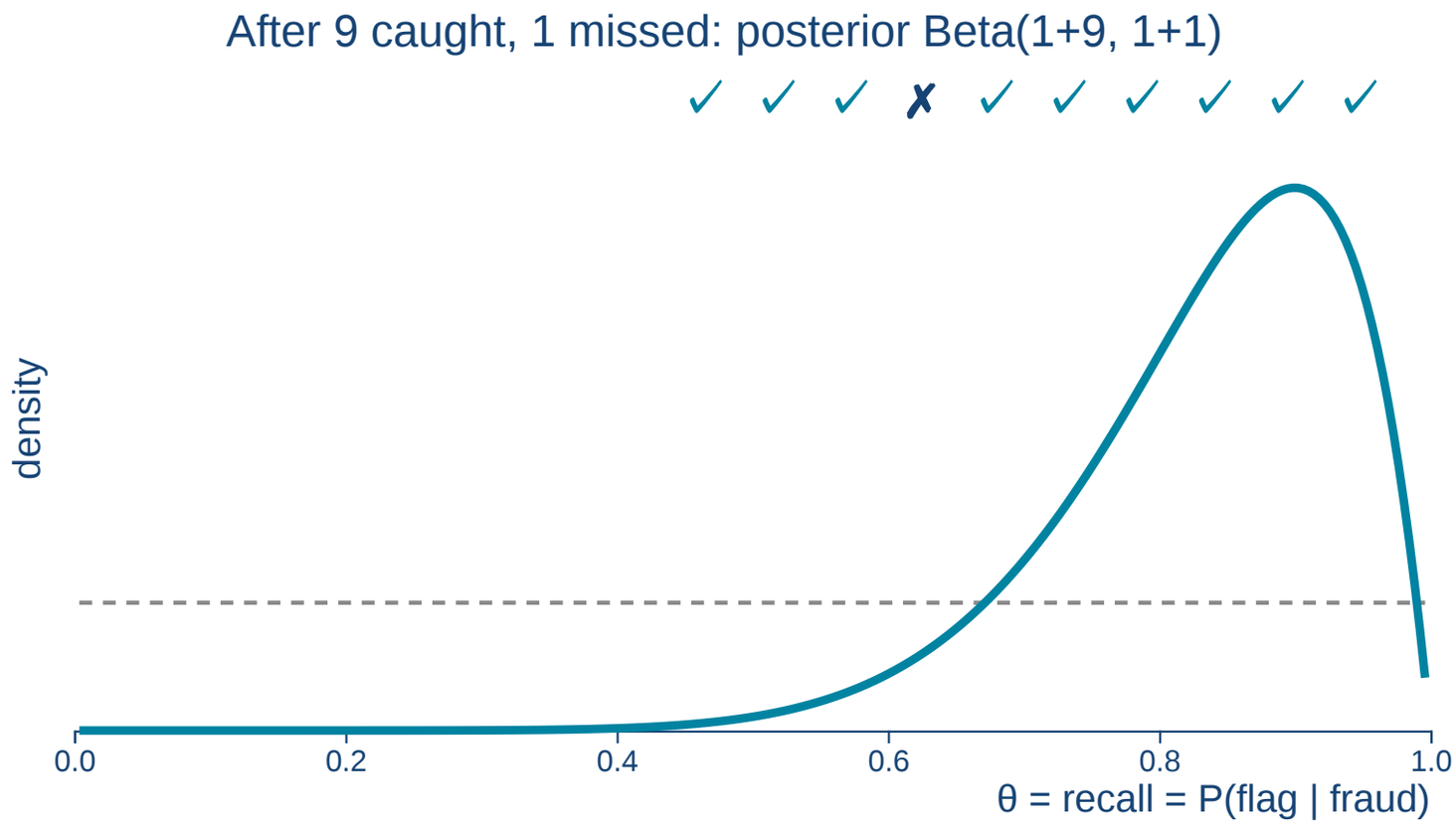
0.90 ± 0.19

recall, 9 of 10 frauds caught: $n = 10$, $\mathrm{SE} \approx 0.095$

Same test set, same model — why is one error bar a hundred times wider?

Recall is a proportion **of the frauds**, and there are only ten of them. The error bar is set by the rare class, not by the size of the test set.

Recall as an Unknown Parameter



After 9 caught and 1 missed, with a flat prior: $\mathrm{Beta}(10, 2)$ — mean **0.83**, 95% credible interval **[0.59, 0.98]**.

Speaker notes

Each test fraud is one Bernoulli observation of $\theta = P(\text{flag} \mid \text{fraud})$: caught = 1, missed = 0. This is the Beta–Bernoulli model of Lecture 03: with prior $\text{Beta}(1, 1)$, k catches out of n give $\text{Beta}(1 + k, 1 + n - k)$. Step 0 of the viz shows the flat prior; step i the posterior after the first i frauds, in the order ✓✓✓✗✓✓✓✓✓✓. Each catch moves the curve right, the single miss (step 4) moves it left, and every observation narrows it a little. After all ten: $\text{Beta}(10, 2)$.

The posterior mean $10/12 = 0.83$ lies below the observed 0.9 because the flat prior acts as one pseudo-catch and one pseudo-miss; the mode, $(10 - 1)/(10 + 2 - 2) = 0.9$, equals the observed proportion. The equal-tailed 95% credible interval $[0.59, 0.98]$ lies inside $[0, 1]$ by construction and is close to the Wilson interval. With 100 test frauds and 90 caught, the posterior $\text{Beta}(91, 11)$ has the interval $[0.83, 0.94]$, about $1/\sqrt{10}$ of the width.

The Rare Class Sets the Test Size

Recall near 0.9, so $\mathrm{SE} = \sqrt{0.09/n_{\mathrm{fraud}}}$; at one fraud in 1,000 transactions:

frauds in the test set	transactions	SE of recall	error bar (±2 SE)
10	10,000	0.095	±0.19
100	100,000	0.030	±0.06
400	400,000	0.015	±0.03
1,000	1,000,000	0.0095	±0.02

Model B catches 10 of the 10 test frauds, model A catches 9. That is **one fraud** of difference — well inside either error bar. Compare two models on the **same** test cases and look at the cases where they disagree.

Speaker notes

With $\hat{p} \approx 0.9$, $\mathrm{SE} = \sqrt{0.09/n_{\text{fraud}}}$: 10 frauds give 0.095, 100 give 0.030, 400 give 0.015, 1,000 give 0.0095. Halving the SE needs four times the frauds (the $1/\sqrt{n}$ law of Lecture 02), and at prevalence 1/1,000 each fraud comes with about 1,000 transactions: an error bar of ±0.02 on recall needs around a million transactions. Fraud evaluations therefore span weeks or months of traffic, during which the fraud pattern itself may drift.

Two recalls measured on separate test sets, each with SE ≈ 0.095, cannot resolve a difference of 0.1. On the same test cases, frauds that both models catch or both miss carry no information about which is better; only the discordant cases do. If the models are equally good, each discordant case goes to either model with probability ½ — the logic of McNemar's test, or an exact sign test on the discordant cases. With B at 10/10 and A at 9/10 there is exactly one discordant fraud; a 1 : 0 split occurs with probability ½ under equality, so nothing can be concluded.

Mini-Exercise

Recall is near 0.9, so $\mathrm{SE} = \sqrt{0.09/n}$ for n test frauds.

How many frauds does the test set need for $\mathrm{SE} = 0.03$ — and how many transactions is that?

Speaker notes

$\mathrm{SE} = 0.03$ requires $0.09/n = 0.03^2 = 0.0009$, so $n = 100$ test frauds; at one fraud in 1,000 transactions that is about 100,000 transactions. Because $n \propto 1/\mathrm{SE}^2$, halving the SE to 0.015 requires 400 frauds (400,000 transactions). The question concerns the SE only; a 95% normal interval would be ±1.96 SE ≈ ±0.06.

The calculation assumes the recall stays near 0.9. The variance $p(1-p)$ is largest at $p = 0.5$ (0.25), so a worst-case plan for SE = 0.03 needs $0.25/0.0009 \approx 278$ frauds.

Mini-Exercise

Recall is near 0.9, so $\mathrm{SE} = \sqrt{0.09/n}$ for n test frauds.

How many frauds does the test set need for $\mathrm{SE} = 0.03$ — and how many transactions is that?

$0.09/n = 0.0009$, so $n = 100$ frauds — about **100,000 transactions** at one in 1,000. For $\mathrm{SE} = 0.015$ it would be four times as many.

Takeaway

> Every metric is an **estimate from a sample** and needs an **error bar**: a standard error, or a Beta posterior when the counts are small. For a rare class the error bar is set by the number of **rare cases**, not by the size of the test set.

Decisions and Scoring Rules

Guiding Question: The model says p(fraud) = 0.09. Check the transaction or let it through — and who decides where the line is?

Two Mistakes, Two Prices

	transaction is fraud	transaction is legitimate
check it (a text to the customer)	0	€5 (c_{FP})
let it through	€500 (c_{FN})	0

The model says $p = p(\text{fraud} \mid \mathbf{x})$. Expected cost of checking: $(1 - p) \cdot c_{FP}$; of letting it through: $p \cdot c_{FN}$.

At which p does checking become the cheaper action?

Speaker notes

Checking costs c_{FP} if the transaction is legitimate (probability $1-p$) and nothing otherwise; letting it through costs c_{FN} if it is a fraud (probability p). Check iff $(1-p)\cdot c_{FP} < p\cdot c_{FN}$, i.e. $p > c_{FP}/(c_{FP}+c_{FN})$; with €5 and €500 this is $5/505 \approx 0.0099$. Only the cost ratio matters. Equal costs give $p^* = 0.5$, so 0.5 is the right threshold only for symmetric costs. The same rule gives the loan threshold of Lecture 01 (€1,000 against €9,000, $p^* = 0.1$).

For the flagged transaction of Lecture 02, $P(\text{fraud} \mid \text{flag}) \approx 0.09$: checking costs $0.91 \cdot €5 \approx €4.55$ in expectation, letting it through $0.09 \cdot €500 = €45$. A low precision is compatible with a correct decision to check; the criterion is expected cost, not precision.

If correct decisions are not free, the rule is: check iff $p\,(c_{FN} - c_{TP}) > (1-p)(c_{FP} - c_{TN})$. If a checked fraud also costs the €5 text, $p^* = 5/500 = 0.01$ instead of $5/505$ — a negligible change.

Two Mistakes, Two Prices

	transaction is fraud	transaction is legitimate
check it (a text to the customer)	0	€5 (c_{FP})
let it through	€500 (c_{FN})	0

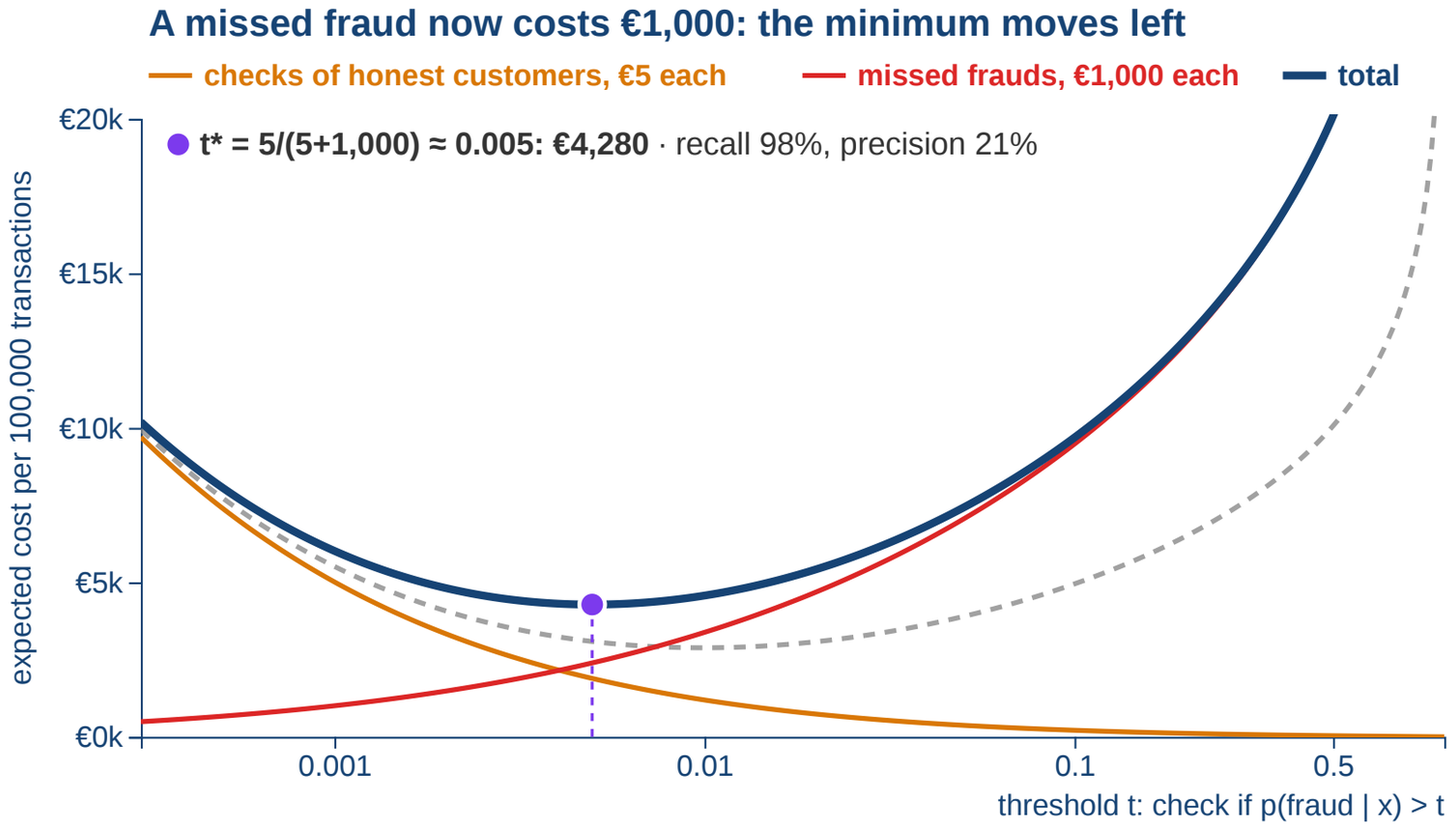
The model says $p = p(\text{fraud} \mid \mathbf{x})$. Expected cost of checking: $(1 - p) \cdot c_{FP}$; of letting it through: $p \cdot c_{FN}$.

At which p does checking become the cheaper action?

$$p > p^* = \frac{c_{FP}}{c_{FP} + c_{FN}} = \frac{5}{505} \approx 0.01$$

> Lecture 01's loan threshold, for fraud. Lecture 02's flagged transaction (9% fraud) is worth checking: 9% is nine times the break-even.

Sweeping the Threshold



Predict before the last click: a missed fraud now costs €1,000. Does the best threshold move left or right — and does recall go up or down?

Speaker notes

The model in the viz is calibrated by construction: a suspicion feature is $\mathcal{N}(0, 1)$ for legitimate and $\mathcal{N}(4.65, 1)$ for fraudulent transactions, prevalence 1/1,000, and the score is the exact posterior $p(\text{fraud} \mid \text{feature})$. Costs are counted over 100,000 transactions (100 frauds).

Step 0: the two cost components — checks of honest customers (€5 each) fall as the threshold rises, missed frauds (€500 each) rise. Step 1: their sum, the expected total cost. Step 2: its minimum at $t^* = 5/505 \approx 0.01$: €2,881, recall 97%, precision 29%. Step 3: two comparison points — $t = 0.001$, which reproduces the flag of Lecture 02 (recall 99%, precision 9%, €5,515), and the default $t = 0.5$ (recall 80%, precision 92%, €10,062, about 3.5 times the minimum). Step 4: with $c_{FN} = €1{,}000$ the minimum moves left to $t^* = 5/1{,}005 \approx 0.005$ (€4,280); the lower threshold checks more transactions, so recall rises to 98% and precision falls to about 21%.

Precision and recall are properties of a model together with a threshold; the costs select the point. The minimum lies exactly at $c_{FP}/(c_{FP} + c_{FN})$ only because the score is calibrated (Lecture 04); for a miscalibrated score the cost-optimal threshold on the score lies elsewhere.

Scoring the Probabilities Themselves

A threshold uses the score once. To evaluate the **probabilities**, score every prediction on held-out data:

Log-loss

$$\text{log-loss} = -\frac{1}{n}\sum_{i=1}^{n} \log p_{\text{model}}(y_i \mid \mathbf{x}_i)$$

the held-out negative log-likelihood — Lecture 04's cross-entropy, on test data

Brier score

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n} \left(p_i - y_i\right)^2$$

squared error of the probability against the 0/1 outcome

Both scores are **proper**: in expectation they are best when the model reports the true probability. Accuracy is not strictly proper — it cannot tell 0.51 from 0.99.

Speaker notes

Log-loss is the average negative log-likelihood of the observed labels under the model's predictive distribution: the Bernoulli negative log-likelihood of Lecture 03 and the cross-entropy that logistic regression minimises in training (Lecture 04), here evaluated on data the model has not seen. On training data it would carry the optimistic bias of any training error.

A scoring rule $S(q, y)$ is proper if the expected score under the true probability p, $\mathbb{E}_{y\sim p}[S(q, y)]$, is optimal at $q = p$, and strictly proper if $q = p$ is the unique optimum. For the Brier score, $\mathbb{E}[(q - y)^2] = p(1 - q)^2 + (1 - p)q^2$ has derivative $2(q - p)$, zero at $q = p$. For log-loss, $-p \log q - (1 - p) \log(1 - q)$ is also minimised at $q = p$. Both are strictly proper: a model cannot improve its expected score by reporting anything other than the probability it believes.

Accuracy uses only the side of 0.5 on which a prediction falls, so 0.51 and 0.99 score the same. Reporting the true p is among its optima, but so is any q on the same side of 0.5: it is not strictly proper and gives no incentive for accurate probabilities.

What the Scores Punish

the model said $p = 0.09$, and the transaction was…	log-loss term	Brier term
fraud	2.41	0.83
legitimate	0.09	0.008
fraud, but the model had said $p = 0.001$	**6.91**	0.998

Log-loss punishes **confident errors** without limit — $p = 0$ on a fraud costs infinity; Brier is bounded by 1. Whether "0.09" comes true 9% of the time is Lecture 04's **reliability diagram**, now drawn on held-out data.

Speaker notes

Per-case terms for $p = 0.09$: on a fraud, $-\ln 0.09 = 2.41$ and $(1 - 0.09)^2 = 0.83$; on a legitimate transaction, $-\ln 0.91 = 0.094$ and $0.09^2 = 0.008$. For $p = 0.001$ on a fraud: $-\ln 0.001 = 6.91$ and $0.999^2 = 0.998$. Going from 0.09 to 0.001 nearly triples the log-loss term but raises the Brier term by only 20%: log-loss grows without bound as the probability of the observed outcome goes to 0, while each Brier term is at most 1.

The always-legitimate model, read as a probabilistic model, assigns $p = 0$ to fraud; one fraud in the test set makes its log-loss infinite. Implementations clip probabilities to $[\varepsilon, 1 - \varepsilon]$, which turns the infinity into a large finite penalty.

A good average score does not by itself establish calibration. Whether cases scored 0.09 are frauds about 9% of the time is checked with the reliability diagram of Lecture 04, computed on held-out data; on training data a flexible model can look calibrated and still be overconfident on new cases.

Mini-Exercise

Checking costs €5, a missed fraud €500. The model says $p = 0.02$.

Check the transaction or let it through?

Speaker notes

Letting the transaction through costs $0.02 \cdot €500 = €10$ in expectation, checking it $0.98 \cdot €5 = €4.90$; checking is cheaper, consistent with $0.02 > p^* = 5/505 \approx 0.0099$. A 2% probability is about twice the break-even at a 100 : 1 cost ratio.

The decision depends on the probability and the cost ratio together. With a missed fraud at €100 (ratio 20 : 1), $p^* = 5/105 \approx 0.048$ and the same transaction would be let through. The structure is the same as in the loan example of Lecture 01: a small probability of an expensive outcome justifies the cautious action.

Mini-Exercise

Checking costs €5, a missed fraud €500. The model says $p = 0.02$.

Check the transaction or let it through?

Check. Letting it through costs 0.02 · €500 = €10 in expectation; checking costs 0.98 · €5 ≈ €4.90. Equivalently, $0.02 > p^* \approx 0.01$.

Takeaway

A **calibrated probability** plus the **costs** gives the decision: act when $p > c_{FP}/(c_{FP} + c_{FN})$. Precision and recall move along one curve as the threshold moves; **log-loss** and the **Brier score** evaluate the probabilities themselves.

Empirical Risk Minimisation

Guiding Question: Every model you trained minimised something on the training data. What did we actually want — and what did the substitution throw away?

What We Actually Want to Minimise

The **risk** of a hypothesis h under a loss L is its expected loss on the world:

$$R(h) = \mathbb{E}_{(\mathbf{x},y)\sim p}\big[L\big(h(\mathbf{x}), y\big)\big]$$

0/1 loss

$R(h) = \mathrm{err}^*(h)$ — the true error of the rare-class module

Cost matrix

$R(h)$ = the expected cost of the decisions module

It is the quantity that matters — and **uncomputable**: p is unknown.

Speaker notes

The risk is defined for any loss L and any distribution p over $(\mathbf{x}, y)$. With the 0/1 loss $L = 1[h(\mathbf{x}) \neq y]$ it is the true error $\mathrm{err}^*(h)$ of the rare-class module; with the fraud cost matrix it is the expected cost per transaction of the decisions module. The minimiser over all functions is the Bayes decision rule for that loss — predict the label with the larger posterior under 0/1 loss, check when $p > c_{FP}/(c_{FP} + c_{FN})$ under the cost matrix — and its risk is the Bayes risk (the Bayes error for 0/1 loss).

The risk is an expectation under the unknown data distribution and cannot be computed. A test set gives an unbiased estimate of $R(h)$ for a fixed h (Uncertainty of Metrics), but training needs a criterion it can optimise directly.

What Training Computes Instead

The **empirical risk** is the average loss on the training sample, and **empirical risk minimisation (ERM)** picks its minimiser:

$$\hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} L\big(h(\mathbf{x}_i), y_i\big), \qquad \hat{h} = \arg\min_{h \in \mathcal{H}} \hat{R}(h)$$

loss	trained model	is $h(\mathbf{x})$ a probability?
squared error	linear regression	the mean of a Gaussian — **yes** (Lecture 04)
cross-entropy $-\log p(y \mid \mathbf{x})$	logistic regression, most neural nets	**yes** — this is maximum likelihood for $p(y \mid \mathbf{x})$
hinge $\max(0, 1 - y \cdot h(\mathbf{x}))$	support vector machine	**no** — a signed margin score
0/1	what we care about; no useful gradient	**no** — only a label

Speaker notes

ERM replaces the expectation under p by the average over the sample (the empirical distribution) and minimises over a hypothesis space $\mathcal{H}$. By the law of large numbers $\hat{R}(h) \to R(h)$ for each fixed h; for the minimiser $\hat{h}$ this is not automatic, which is the generalisation gap.

The rows: least squares is maximum likelihood under Gaussian noise, and $h(\mathbf{x})$ is the mean of the predictive Gaussian (Lecture 04). Cross-entropy is ERM with $L = -\log p(y \mid \mathbf{x})$, identical to conditional maximum likelihood, so the output is a probability. The expected hinge loss is minimised by $\operatorname{sign}\big(2p(y{=}1 \mid \mathbf{x}) - 1\big)$: it targets the 0/1 decision, and the score $\mathbf{w}^\top \mathbf{x} + b$ is the signed distance to the decision boundary times $\|\mathbf{w}\|$, not a probability. The 0/1 loss is piecewise constant in the parameters, so its gradient is zero almost everywhere, and minimising the empirical 0/1 loss is NP-hard in general, even for linear classifiers; cross-entropy and hinge are convex surrogates for it.

ERM is one recipe among several. The Beta posterior (Lecture 03), inference in a Bayesian network (Lecture 05) and Bayesian linear regression (Lecture 07) condition on the data; nothing is minimised.

Keep the Loss, Drop the Probabilities

WHAT ERM WITH A HINGE OR 0/1 LOSS RETURNS

a score: 2.3

large means "more fraud-like" — nothing more

WHAT IT CAN NO LONGER DO

- apply the cost threshold $p > c_{FP}/(c_{FP} + c_{FN})$ of the decisions module
- be scored by log-loss or Brier, or drawn in a reliability diagram
- put an error bar on its own confidence

To use it for a decision, **buy the probabilities back**: fit $p = \sigma(a \cdot \text{score} + b)$ on held-out data (Platt scaling) — a logistic regression on the score, i.e. calibration.

Not every method is ERM: maximum likelihood for $p(y \mid \mathbf{x})$ is ERM with loss $-\log p$; generative maximum likelihood (naive Bayes) scores $\log p(\mathbf{x}, y)$, and Bayesian methods minimise nothing.

Speaker notes

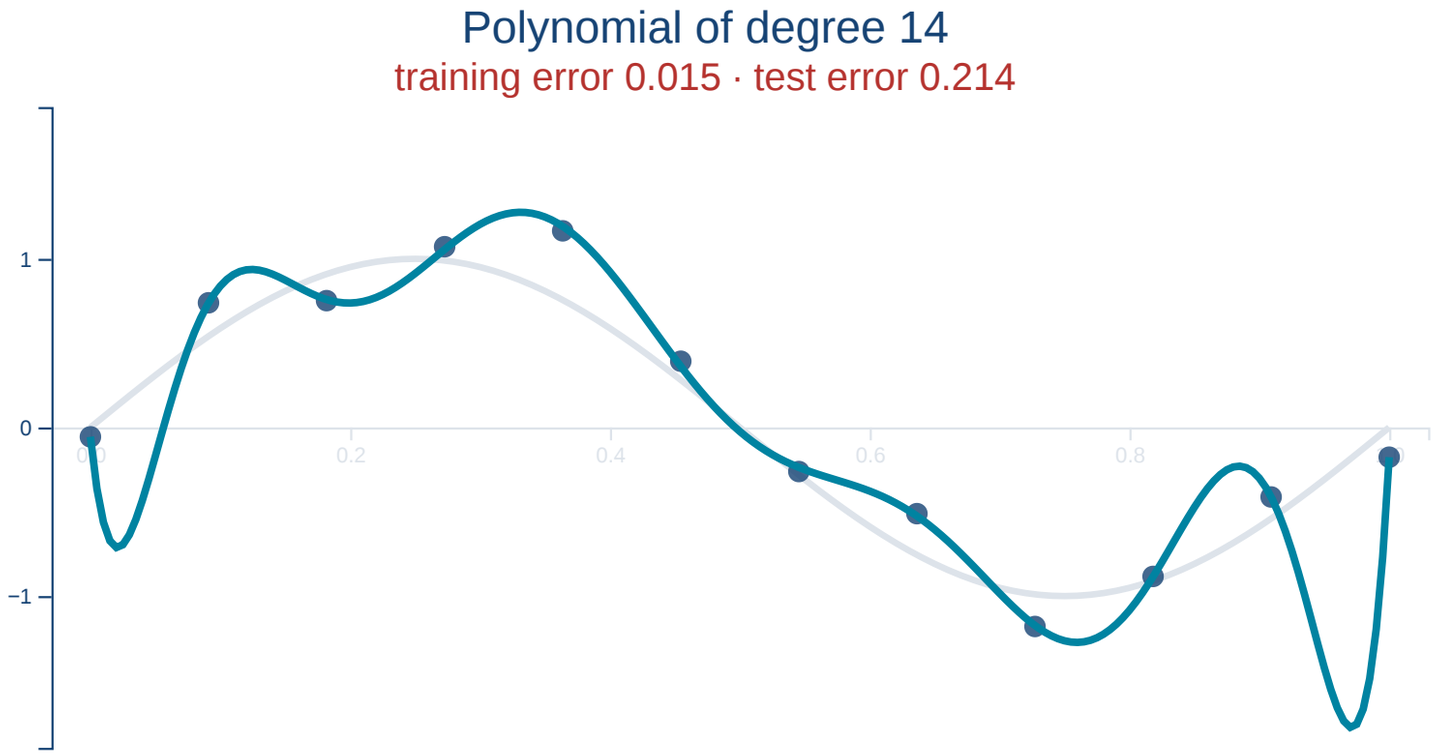
An SVM score of 2.3 means that the transaction lies on the fraud side of the boundary and beyond the margin ($\mathbf{w}^\top \mathbf{x} + b > 1$). Without a map from scores to probabilities it says nothing about $p(\text{fraud} \mid \mathbf{x})$, so the cost rule, log-loss, the Brier score and the reliability diagram cannot be applied to it.

Platt scaling fits $p = \sigma(a \cdot s + b)$ to pairs (score, label) by maximum likelihood — a logistic regression with the score as its only feature (Lecture 04). It is fitted on data the SVM was not trained on: on training data the scores are optimistic and the fitted map becomes overconfident. Isotonic regression is the non-parametric alternative when the score-to-probability map is not sigmoid-shaped and enough calibration data are available.

Training with cross-entropy does not guarantee calibration on new data: large neural networks trained to near-zero training loss are often overconfident (Guo et al., 2017), so the reliability check applies to them as well.

Naive Bayes fitted by maximum likelihood maximises $\sum_i \log p(\mathbf{x}_i, y_i)$, a joint likelihood that also models the inputs; it is not an average of a loss $L(h(\mathbf{x}_i), y_i)$ on predictions.

The Generalisation Gap



$R(\hat{h}) = \hat{R}(\hat{h}) + \big(R(\hat{h}) - \hat{R}(\hat{h})\big)$: the gap opens because $\hat{h}$ was **chosen on the data that then measures it**, and it tends to grow with the capacity of $\mathcal{H}$.

Speaker notes

The viz fits least-squares polynomials of degree 1, 2, 3, 5, 9 and 14 (one per step, steps 0–5) to the same twelve noisy samples of $\sin(2\pi x)$ and reports the RMSE on the training points and on a second sample of twelve test points. Training error falls at every step (0.55 at degree 1, 0.015 at degree 14). Test error is lowest at degree 5 (about 0.16) and rises again at degrees 9 and 14 (about 0.21). Degrees 1 and 2 cannot follow a full sine period, so both errors are high there.

For a fixed h chosen independently of the sample, $\hat{R}(h)$ is an unbiased estimate of $R(h)$; the bias belongs to the selected $\hat{h}$, because selection favours hypotheses that happen to fit this sample well. A richer $\mathcal{H}$ can widen the gap and still lower the risk. With $h^*_{\mathcal{H}}$ the best hypothesis in $\mathcal{H}$ and R^* the Bayes risk,

$$R(\hat{h}) - R^* = \underbrace{R(\hat{h}) - R(h^*_{\mathcal{H}})}_{\text{estimation error}} + \underbrace{R(h^*_{\mathcal{H}}) - R^*}_{\text{approximation error}}$$

Approximation error falls as capacity grows; estimation error, for fixed n, tends to grow. This decomposition, not the training–test gap, is the risk-level counterpart of the bias–variance decomposition.

Structural Risk Minimisation — and the Prior Inside It

Minimise the empirical risk **plus** a complexity penalty:

$$\hat{h} = \arg\min_{h \in \mathcal{H}} \hat{R}(h) + \lambda \Omega(h)$$

With the loss $-\log p(y \mid \mathbf{x}, \mathbf{w})$, the MAP estimate of Lecture 04 has exactly this form:

$$\frac{1}{n} \sum_{i=1}^{n} -\log p(y_i \mid \mathbf{x}_i, \mathbf{w}) \quad \underbrace{-\frac{1}{n} \log p(\mathbf{w})}_{\lambda \Omega(\mathbf{w})}$$

> The $\frac{1}{n}$ is the point: the prior's weight **fades as data grows** — the pseudo-counts of Lecture 03's Beta prior. A Gaussian prior $\mathcal{N}(0, \tau^2 \mathbf{I})$ gives $\lambda \Omega(\mathbf{w}) = \|\mathbf{w}\|^2 / (2n\tau^2)$.

Speaker notes

The MAP estimate maximises $\log p(\mathbf{w} \mid D) = \sum_i \log p(y_i \mid \mathbf{x}_i, \mathbf{w}) + \log p(\mathbf{w}) + \text{const}$. Negating and dividing by n, which does not move the optimum, gives the average loss plus $-\frac{1}{n} \log p(\mathbf{w})$. For $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I})$, $-\log p(\mathbf{w}) = \|\mathbf{w}\|^2 / (2\tau^2) + \text{const}$, so $\lambda \Omega(\mathbf{w}) = \|\mathbf{w}\|^2 / (2n\tau^2)$: weight decay whose strength falls as $1/n$. A Laplace prior gives the L_1 penalty in the same way.

Penalty, capacity control and prior are three readings of one term. The $1/n$ explains why regularisation matters for small samples and fades for large ones, as the pseudo-counts of a Beta prior (Lecture 03) are swamped by data.

MAP still returns a single parameter vector. It controls the generalisation gap but carries no uncertainty about $\mathbf{w}$; predictions that reflect parameter uncertainty need the full posterior (Lecture 07). In Vapnik's original formulation, structural risk minimisation chooses among nested hypothesis spaces by minimising a bound on the risk; the penalised objective is its common practical form.

Mini-Exercise

A linear SVM gives a transaction the score **2.3**. The fraud team checks every transaction with $p > 0.01$.

Can they apply their rule to this transaction?

Speaker notes

The score 2.3 is $\mathbf{w}^\top \mathbf{x} + b$, a scaled signed distance to the decision boundary; comparing it with 0.01 compares quantities on different scales. The SVM's own decision, fraud iff the score is positive, approximates the minimiser of the expected hinge loss, $\operatorname{sign}(2p - 1)$ — the threshold $p = 0.5$ of equal costs, not the bank's $p^* \approx 0.01$.

Applying the bank's rule requires a calibration map $p = \sigma(a \cdot s + b)$ fitted on held-out data (Platt scaling), then the comparison $p > 0.01$. Two frequent errors: reading the score as a probability ("2.3 > 0.01, check"), and taking the sign of the score as the decision, which silently assumes equal costs.

Mini-Exercise

A linear SVM gives a transaction the score **2.3**. The fraud team checks every transaction with $p > 0.01$.

Can they apply their rule to this transaction?

Not directly. 2.3 is a margin score, not a probability — the hinge loss dropped it. First calibrate on held-out data, $p = \sigma(a \cdot 2.3 + b)$, then compare that p with 0.01.

Takeaway

Training minimises the **empirical risk** because the **risk** is out of reach. ERM keeps the **loss** and — unless the loss is a negative log-probability — drops the **probabilities**. Regularisers (penalties or priors) control the gap; error bars measure it; calibration buys the probabilities back.